

Initiation à Scilab

Version Matlab par Florence Hubert
adaptation à Scilab par Franck Boyer

Table des matières

1	Première utilisation de Scilab, l'aide en ligne	3
2	Les matrices dans Scilab	4
2.1	Création de matrices	4
2.2	Matrices et vecteurs	6
2.3	Les premières opérations sur les matrices	7
2.4	Matrices et tableaux	8
3	Le calcul vectoriel	8
4	Programmation en Scilab	9
4.1	Les fonctions élémentaires	9
4.2	Créer une fonction ou un script	9
4.2.1	Les fonctions	9
4.2.2	Bibliothèque de fonctions	10
4.2.3	Remarque importante	11
4.2.4	Les scripts	11
4.2.5	Exécuter un script par Scilab	12
4.2.6	Syntaxe des opérations logiques	13
4.2.7	Syntaxe des boucles	13
4.2.8	Structure de choix	14
4.3	Conseils de programmation	14
4.4	Enregistrer ou lire des données créées par Scilab	15
5	Graphisme dans Scilab	16
5.1	Graphe de points en 2D	16
5.2	Graphe d'une fonction	16
5.3	Graphe d'une courbe dans l'espace	16
5.4	Graphe d'une surface	16
5.5	Graphe d'une fonction de deux variables réelles ou d'une surface paramétrée	17
5.6	Gestion des figures	18
5.6.1	Ouvrir une figure	18
5.6.2	Ouvrir plusieurs figures	18
5.6.3	La commande <code>clf</code>	18
5.6.4	Créer plusieurs graphes sur une même figure	18
5.6.5	Sauvegarde et export d'une figure	18
5.7	Animation de graphisme	18
6	Quelques commandes utiles	19
6.1	Opérations sur les coefficients d'une matrice	19
6.2	Propriétés élémentaires d'une matrice	19
6.3	Norme et conditionnement d'une matrice	19
6.4	Réduction classiques pour les matrices	19

6.5	Interpolation numérique	20
6.6	Intégration numérique	20
6.7	Equations différentielles ordinaires	20
6.8	Equations aux dérivées partielles	20
6.9	Algorithmes de minimisation et de recherche de racines	20
6.10	Zéros de fonctions	20
6.11	Fourier	20

Introduction

Scilab est un programme de calcul numérique. La grande majorité des objets en jeu dans Scilab sont des matrices, y compris les scalaires (matrices 1×1).

Avec quelques petits efforts, vous pourrez (en utilisant Scilab) mettre en oeuvre les différentes méthodes que vous connaissez (ou que vous ne tarderez pas à connaître) pour :

- Résoudre des systèmes linéaires du type $Ax = b$ ou calculer des valeurs propres et vecteurs propres de matrices.
- Calculer des valeurs approchées d'intégrales.
- Résoudre des équations non-linéaires du type $f(x) = 0$.
- Calculer des solutions approchées d'équations différentielles ordinaires.
- Calculer des solutions approchées d'équations aux dérivées partielles.
- Tracer les solutions des problèmes ci-dessus de la manière la plus propre possible pour que les résultats soient exploitables. On pourra même faire afficher une animation montrant l'évolution de la solution en fonction du temps.
- Et bien d'autres choses

Nous proposons ici des rappels non exhaustifs des commandes utiles en calcul scientifique. Les exemples donnés ont été effectués avec Scilab 4.1.1 mais doivent fonctionner à l'identique dans les versions plus récentes.

1 Première utilisation de Scilab, l'aide en ligne

Une fois Scilab lancé, les instructions que l'utilisateur va donner à Scilab doivent suivre le sigle `->`.

Les instructions ainsi données au logiciel sont exécutées (ou pas s'il y a une erreur de syntaxe !) au moment où vous validez la ligne de commandes avec la touche *ENTREE*.

Par exemple :

```
-->1+1
ans  =

    2.

-->%pi
%pi  =

    3.1415927

-->%eps
%eps =

    2.220E-16

-->%i^2
ans  =

    - 1.

-->t=1+1
t    =

    2.

-->t=1+1;

-->t
t    =

    2.
```

```

-->u=sin(t)
u =

    0.9092974

-->v=exp(u)
v =

    2.4825777

-->format('e',20);

-->v
v =

    2.4825777280150E+00

-->format('v',6);

-->v
v =

    2.48258

-->exit

```

On remarque que :

- Par défaut, tout calcul est affecté dans la variable `ans` (pour « answer » en anglais).
- Les variables `%pi`, `%eps`, `%i` sont déjà affectées et ne peuvent être réaffectées.
- L'affectation se fait grâce au sigle `=`.
- Le résultat d'une affectation est imprimé sauf si cette affectation est suivie du sigle `;`.
- La commande `format` permet de modifier l'affichage du format des différentes variables.
- La commande `clear` efface le contenu de toutes les variables utilisées.

Attention : il est très fortement déconseillé d'utiliser des noms de variables déjà utilisées par Scilab.

Pour sortir de Scilab, il suffit d'utiliser les commandes `quit` ou `exit`. Lorsque l'on veut interrompre un programme, la commande `CTRL-c` permet de récupérer la main.

Une façon efficace de découvrir Scilab est d'utiliser **son aide en ligne**.

- `help` : "help" tout seul donne la liste des aides générales possibles.
- `help nom de commande` : exemple, `help plot` indique la syntaxe des graphes en 2D.
- `apropos nom de commande` : exemple, `apropos plot` donne une liste de toutes les commandes qui ont un rapport avec `plot`.

Attention, il n'est pas possible de sauvegarder la *fenêtre de commande* de Scilab. Pour sauvegarder un travail, il est conseillé de travailler dans une fenêtre d'édition. On peut utiliser l'éditeur de Scilab et ouvrir ou créer un fichier en cliquant sur **File** puis **Open** ou **New**. Tout autre éditeur de texte (Emacs, Nedit,...) convient également. Pour exécuter les commandes d'un fichier qui s'appelle `toto.sce`, il suffit alors de taper `exec toto.sce` ou `exec 'toto.sce'` dans la fenêtre de commandes de Scilab.

2 Les matrices dans Scilab

2.1 Création de matrices

- $A = [a_{11} \dots a_{1m}; \dots; a_{n1} \dots a_{nm}]$
- Exemple :

```

-->A=[1 2 3 4;2 3 4 1;3 4 1 2;4 1 2 3]
A =

    1.    2.    3.    4.

```

```

    2.    3.    4.    1.
    3.    4.    1.    2.
    4.    1.    2.    3.
- Quelques matrices prédéfinies

```

```

-->B=zeros(2,3)
B =

```

```

    0.    0.    0.
    0.    0.    0.

```

```

-->C=ones(3,2)
C =

```

```

    1.    1.
    1.    1.
    1.    1.

```

```

-->Id=eye(2,3)
Id =

```

```

    1.    0.    0.
    0.    1.    0.

```

```

-->D=rand(4,4)
D =

```

```

    0.8784126    0.5896177    0.3493615    0.3435337
    0.1138360    0.6853980    0.3873779    0.3760119
    0.1998338    0.8906225    0.9222899    0.7340941
    0.5618661    0.5042213    0.9488184    0.2615761

```

```

-->E=rand(3,4,'normal')
E =

```

```

- 1.7350313 - 2.0735088 - 0.4534087 - 1.2317066
  0.5546874   2.5891773 - 1.6565358 - 0.1325739
- 0.2143931   0.0424792   1.7224151 - 1.7242355

```

`zeros(n,m)` construit une matrice nulle de taille $n \times m$,

`ones(n,m)` construit une matrice dont tous les éléments sont égaux à un et de taille $n \times m$,

`eye(n,m)` construit une matrice de taille $n \times m$ dont tous les éléments diagonaux sont égaux à un,

`rand(n,m)` construit une matrice de taille $n \times m$ dont tous les éléments sont choisis aléatoirement avec la loi uniforme sur $[0, 1]$,

`rand(n,m,'normal')` construit une matrice de taille $n \times m$ dont tous les éléments sont choisis aléatoirement avec la loi normale centrée réduite.

– Construction élément par élément

Exemple :

```

-->for i=1:3,for j=1:4, F(i,j)=i+(j-1)*3;end;end;

```

```

-->F
F =

```

```

    1.    4.    7.   10.
    2.    5.    8.   11.
    3.    6.    9.   12.

```

– La commande `size` permet d'obtenir la taille de la matrice :

```

-->size(A)

```

```

ans =

    4.    4.

-->size(F)
ans =

    3.    4.

-->size(F,1)
ans =

    3.

-->size(F,2)
ans =

    4.

```

2.2 Matrices et vecteurs

Par défaut, les vecteurs dans Scilab sont écrits en colonne :

```

-->for i=1:3, V(i)=i;end;

-->V
V =

    1.
    2.
    3.

-->size(V)
ans =

    3.    1.

```

Une matrice $n \times m$ est stockée colonnes après colonnes dans Scilab comme un vecteur de taille $n m$:

```

-->for i=1:3,for j=1:4, F(i,j)=i+(j-1)*3;end;end;

-->F
F =

    1.    4.    7.   10.
    2.    5.    8.   11.
    3.    6.    9.   12.

-->F(4)
ans =

    4.

-->F(8)
ans =

    8.

```

De même, on peut affecter directement les éléments d'une matrice, après avoir défini sa taille :

```
-->F=zeros(3,4);

-->for i=1:12,F(i)=i;end

-->F
F =

    1.    4.    7.   10.
    2.    5.    8.   11.
    3.    6.    9.   12.
```

2.3 Les premières opérations sur les matrices

- Transposition :

```
-->F'
ans =

    1.    2.    3.
    4.    5.    6.
    7.    8.    9.
   10.   11.   12.
```

- La diagonale, la trace, le rang, le déterminant

```
-->diag(A)
ans =

    1.
    3.
    1.
    3.
```

```
-->trace(A)
ans =

    8.
```

```
-->rank(A)
ans =

    4.
```

```
-->det(A)
ans =

   160.
```

- Addition ou soustraction

```
-->F+E
ans =

    0.3714058    4.3247476    7.4620729   10.844049
    0.3814665    4.7792285    7.1368225    9.3313411
    2.5659409    5.7027756    9.4870404   10.389694
```

- Multiplication, la puissance $n^{ième}$

```
-->F*A
ans =
```

```

70.    52.    46.    52.
80.    62.    56.    62.
90.    72.    66.    72.

-->A^3
ans =

240.    244.    256.    260.
244.    256.    260.    240.
256.    260.    240.    244.
260.    240.    244.    256.
- L'inverse

-->inv(A)
ans =

- 0.225    0.025    0.025    0.275
0.025    0.025    0.275    - 0.225
0.025    0.275    - 0.225    0.025
0.275    - 0.225    0.025    0.025

```

2.4 Matrices et tableaux

Une matrice peut être considérée élément par élément, c'est alors un tableau. Les opérations élémentaires élément par élément sur les tableaux sont alors les suivantes :

- Addition ou soustraction : pas de changements.
- Multiplication, la puissance $n^{ième}$

```

-->D.*A
ans =

0.8784126    1.1792355    1.0480846    1.3741349
0.2276719    2.0561939    1.5495115    0.3760119
0.5995013    3.5624899    0.9222899    1.4681881
2.2474643    0.5042213    1.8976369    0.7847284

-->A.^3
ans =

1.    8.    27.    64.
8.    27.    64.    1.
27.    64.    1.    8.
64.    1.    8.    27.

```

Remarque 2.1 Si les tailles des tableaux ou matrices ne sont pas adaptées, Scilab donne des messages d'erreurs de la forme :

```

-->F.*A
!--error 9999
inconsistent element-wise operation

```

3 Le calcul vectoriel

Il est souvent intéressant de travailler sur les lignes ou les colonnes d'une matrice avec Scilab.

- $A(i, :)$ désigne la $i^{ième}$ ligne de la matrice A
- $A(2:3, :)$ désigne la sous matrice formée des $2^{ième}$ et $3^{ième}$ lignes de la matrice A
- $A(:, j)$ désigne la $j^{ième}$ colonne de la matrice A
- $A(:, 1:2:m)$ désigne la sous matrice formée des colonnes impaires de la matrice A

Remarque 3.1

- `xmin:dx:xmax` désigne l'ensemble des points de la forme `xmin + i dx`, pour $i \in \mathbb{N}$, qui compris entre `xmin` et `xmax`, l'extrémité `xmin` étant toujours incluse. Par exemple, `1:3:7` ou `1:3:9` désignent le vecteur `[1,4,7]`
- `xmin : xmax` idem avec par défaut `dx=1`.
- La commande `linspace` permet de définir des progressions arithmétiques, autrement dit des découpages d'un intervalle de $\mathbb{R}$ en un certain nombre d'intervalles de longueurs égales. Exemple :

```
-->linspace(0,1,10)
ans =
```

```
column 1 to 7
```

```
0.    0.1111111  0.2222222  0.3333333  0.4444444  0.5555556  0.6666667
```

```
column 8 to 10
```

```
0.7777778  0.8888889  1.
```

Remarquez que le résultat contient 10 points et donc 9 intervalles.

4 Programmation en Scilab

4.1 Les fonctions élémentaires

Un certain nombre de fonctions élémentaires sont prédéfinies par `exp`, `sin`, `cos`, `abs`, ... La plupart de ces fonctions agissent également sur des matrices ou vecteurs :

```
-->x=[0:0.5:%pi]
x =
```

```
0.    0.5    1.    1.5    2.    2.5    3.
```

```
-->sin(x)
ans =
```

```
0.    0.4794255  0.8414710  0.9974950  0.9092974  0.5984721  0.1411200
```

4.2 Créer une fonction ou un script

On va créer un fichier « texte » que l'on appellera `nomdefonction.sci` ou un script que l'on appellera `nomdescript.sce`. Vous pouvez utiliser l'éditeur de Scilab (cliquer en haut à droite sur Editor) mais aussi, n'importe quel éditeur de texte. **emacs**, **nedit**, **Text editor**, ...

On va décrire la syntaxe de ces fichiers à l'aide de deux exemples.

- Pour charger en mémoire une fonction définie dans `nomdefonction.sci` on tape la commande

```
--> exec 'nomdefonction.sci';
```

On peut alors disposer de la fonction définie dans le fichier comme toute autre fonction de Scilab.

- Pour exécuter les commandes contenues dans un script `nomdescript.sce` on tape la commande

```
--> exec 'nomdescript.sce';
```

4.2.1 Les fonctions

On va créer une fonction $f : (x, l) \rightarrow \frac{1}{x^2 + l}$. Le premier exemple est édité sous le nom `f1.sci`.

```
function [y]=f1(x,l)
y=1/(1+x^2);
// on peut agrémente tous ces fichiers par des
```

```
// commentaires qui seront précédés du sigle //
endfunction
```

Une autre possibilité permet de travailler avec des variables x qui seront des tableaux (fichier `f2.sci`) :

```
function [y]=f2(x,l)
    y=ones(x)./(1+x.^2);
    // Les opérateurs agissant sur les éléments des tableaux
    // sont les opérateurs classiques précédés d'un point
    // on utilise la commande ones(x) pour créer un tableau
    // qui ne contient que des 1 de la même taille que x
endfunction
```

Avec les exemples ci-dessus essayer les instructions suivantes

```
--> f1(1,1)
--> f2(1,1)
--> f1(1,[1,2])
--> f1(1,[1,2]')
--> f2(1,[1,2])
--> f2(1,[1,2]')
--> f1([1,2],[2,1])
--> f2([1,2],[2,1])
--> f2([1,2,3],[1,2])
```

Remarque 4.1 Il est également possible de définir des fonctions *simples* grâce à la commande `deff` comme le montre l'exemple qui suit.

```
deff(' [y]=f(x)', 'y=sin(2*x)+x^2');
```

4.2.2 Bibliothèque de fonctions

On peut mettre plusieurs fonctions dans un même fichier `nomdebibliotheque.sci` et les charger toutes simultanément grâce à la commande `exec`. Le nom du fichier peut être quelconque et sans lien avec les noms des fonctions qu'il définit.

Exemple : contenu d'un fichier `mesfonctions.sci`

```
function [y]=f1(x,l)
    y=1/(1+x^2);
endfunction

function [z]=f2(x,y)
    z=ones(x)./(y+x.^2);
endfunction
```

Remarque 4.2 Notez bien que les noms des variables utilisées en entrée et en sortie d'une fonction n'ont aucune importance sur l'action d'une fonction. Ces variables sont **muettes**, et elles sont seulement définies à l'intérieur de la fonction ; elles sont inconnues du programme qui les appelle.

ATTENTION : Quand vous modifiez les définitions des fonctions dans une bibliothèque de fonctions, il ne faut pas oublier de recharger la bibliothèque avec la commande `exec` sinon les modifications ne seront pas prises en compte.

Scilab vous prévient d'ailleurs qu'une fonction a changé par un message du type

```
Warning :redefining function: f
```

4.2.3 Remarque importante

Dans Scilab, les fonctions sont des variables comme les autres. En particulier, on peut passer une fonction en paramètre d'une autre fonction.

Par exemple, la fonction suivante calcule une approximation de l'intégrale d'une fonction **f** sur un intervalle $[a, b]$ par la méthode des rectangles à gauche :

```
function [I]=integrale(f,a,b,N)
    x=linspace(a,b,N+1);
    h=x(2:N+1)-x(1:N);
    I=sum(h.*f(x(1:N)));
endfunction
```

Si maintenant on définit les différentes fonctions suivantes

```
function [y]=f1(x)
    y=cos(x);
endfunction
```

```
function [y]=f2(x)
    y=sin(x);
endfunction
```

On peut utiliser la fonction **integrale** à ces deux fonctions comme suit

```
-->integrale(f1,0,1,100)
ans =
```

```
0.8437625
```

```
-->integrale(f2,0,1,100)
ans =
```

```
0.4554865
```

4.2.4 Les scripts

Quand on définit une fonction et qu'on la charge dans Scilab, aucun calcul n'est effectué. On ne fait que donner à Scilab des définitions de fonctions qu'il utilisera par la suite. Les calculs ne sont effectués que lors de l'appel des fonctions.

A *contrario* un script (ou programme) est une suite d'instructions Scilab qui vont être exécutées immédiatement par le logiciel lors du chargement du script en question.

En général, un script utilise un certain nombre de fonctions définies par l'utilisateur et rassemblées dans une bibliothèque de fonctions (i.e. un fichier **.sci**). C'est pourquoi l'une des toutes premières lignes d'un script correspond très souvent au chargement d'une telle bibliothèque.

Exemple : On se donne une discrétisation de l'intervalle $[0, 1]$ de pas h , et on crée un script qui calcule les vecteurs X et F de composantes $X(i) = (i - 1) \times h$ et $F(i) = f((i - 1) \times h, l)$, où $f(x, l) = \frac{1}{l+x^2}$.

Le premier programme est sauvegardé sous le nom **essai1.sce** et fait appel à la fonction **f1** qui est définie dans la librairie **mesfonctions.sci** introduite au paragraphe 4.2.2.

```
//essai1.sce
exec mesfonctions.sci; // On charge les fonctions dont on a besoin

l=1;
h=0.1;
N=floor(1/h); // partie entière
for i=1:N+1, X(i)=(i-1)*h;end
for i=1:N+1, F(i)=f1((i-1)*h,l);end
```

```
disp(h)
disp(X,'valeur de X =')
F
```

Le deuxième programme est sauvegardé sous le nom `essai2.sce`, fait appel à la fonction `f2` qui se trouve dans la même librairie et utilise le calcul vectoriel, la variable `h` est laissée comme paramètre à choisir ultérieurement. Si on ne donne pas de valeur à cette variable avant l'exécution du script, Scilab arrêtera l'exécution et retournera un message d'erreur.

```
//essai2.sce
exec mesfonctions.sci;

l=1;
X=[0:h:1];
F=f2(X,l);
X
F
```

Remarque 4.3

- Les commentaires situés juste après la première ligne d'une fonction `lambda.sci` sont accessibles dans Scilab par la commande `help lambda`.
- On peut déclarer une fonction à l'intérieur d'un script à l'aide de la syntaxe `function`. On peut également emboîter les définitions de fonctions.

4.2.5 Exécuter un script par Scilab

Il faut utiliser la commande `exec` suivi du nom du script et d'un point-virgule (si on ne veut pas que les commandes s'affichent à l'écran).

```
-->exec test2.sce;
```

```
0.1
```

```
valeur de X=
```

```
0.
```

```
0.1
```

```
0.2
```

```
0.3
```

```
0.4
```

```
0.5
```

```
0.6
```

```
0.7
```

```
0.8
```

```
0.9
```

```
1.
```

```
ans =
```

```
1.
```

```
0.9900990
```

```
0.9615385
```

```
0.9174312
```

```
0.8620690
```

```
0.8
```

```
0.7352941
```

```
0.6711409
```

```
0.6097561
```

```
0.5524862
```

```
0.5
```

Remarquez les différences entre les trois façons proposées pour afficher des variables en utilisant (ou pas) la commande `disp`.

En utilisant l'autre script `essai2.sce`, comme h est laissé en paramètre dans le programme, on lance le calcul de la façon suivante

```
-->h=0.1;

-->exec test3.sce;

0.1

valeur de X=

      column 1 to 9
0.    0.1    0.2    0.3    0.4    0.5    0.6    0.7    0.8

      column 10 to 11
0.9    1.
ans =

      column 1 to 6
1.    0.9900990    0.9615385    0.9174312    0.8620690    0.8

      column 7 to 11
0.7352941    0.6711409    0.6097561    0.5524862    0.5
```

Notons que dans ce programme, X est un vecteur ligne et donc F aussi. Malgré cette différence, les résultats sont bien sûr les mêmes que précédemment.

4.2.6 Syntaxe des opérations logiques

- `==` égalité
- `>=` supérieur ou égal
- `~=` différent
- `&` « et » logique
- `|` « ou » logique
- `~` la négation

Les valeurs *vrai* (en anglais : *TRUE*) et *faux* (en anglais : *FALSE*) correspondent aux variables prédéfinies `%T` et `%F`.

4.2.7 Syntaxe des boucles

- `for i=imin : imax`
`< commandes >;`
`end`
- ou
- `for i=imin : h : imax`
`< commandes >;`
`end`

Exemples (remarquez la virgule après le `imax` dans le cas où on met tout sur une seule ligne) :

```
-->for i=1:2:10,X(i)=i;end
```

```

-->X
X =

    1.
    0.
    3.
    0.
    5.
    0.
    7.
    0.
    9.

-->for i=10:-1:1,Y(i)=10-i;end

-->Y
Y =

    9.
    8.
    7.
    6.
    5.
    4.
    3.
    2.
    1.
    0.
- while < test >,
    < commandes >
end
- if < test >,
    < commandes >
elseif < test >,
    < commandes >
else < commandes >,
end

```

4.2.8 Structure de choix

```

select < nomdevariable >
case < valeur1 > then
    < commandes à exécuter si la variable vaut exactement valeur1 >
case < valeur2 > then
    < commandes à exécuter si la variable vaut exactement valeur2 >
case < valeur3 > then
    < commandes à exécuter si la variable vaut exactement valeur3 >
:
:
else
    < commandes à exécuter dans tous les autres case >
end

```

4.3 Conseils de programmation

Pour gagner du temps en Scilab, il faut éviter le plus possible les grandes boucles dans les programmes. Illustrons ce fait sur un exemple :

```

// test.sce
N=1000;

```

```

a=1000*ones(N,N);
z=zeros(N,N);
tic // initialise le temps
for i=1:N, for j=1:N, z(i,j)=a(i,j); end; end;
toc // affiche le temps écoulé depuis le dernier tic

```

```

tic
z=a;
toc

```

qui donne à l'exécution

```

-->exec test.sce;
ans =

    7.599
ans =

    0.014

```

Par ailleurs, il est toujours bon d'initialiser les tableaux à 0 (ou toute autre valeur) en indiquant à Scilab la bonne taille du tableau.

Par exemple, le script suivant (semblable au précédent mais sans l'initialisation de la variable *z*)

```

// test2.sce
clear // On efface toutes les variables de la mémoire
N=1000;
a=1000*ones(N,N);
tic
for i=1:N, for j=1:N, z(i,j)=a(i,j); end; end;
toc

```

donne le résultat suivant

```

-->exec test2.sce;
ans =

   18.308

```

à comparer aux résultats obtenus avec le script précédent.

4.4 Enregistrer ou lire des données créées par Scilab

- `save('nomdefichier')` :
sauve toutes les variables existantes dans le fichier `nomdefichier`.
- `save('nomdefichier',F,X)`
sauve les variables *F* et *X* dans le fichier `nomdefichier`.
Le format du fichier obtenu est dit *binnaire*, c'est-à-dire qu'il n'est pas lisible dans un éditeur de texte. Il ne peut être utilisé qu'avec la commande Scilab `load` (voir plus loin).
- `nomdufichier='ah'+string(h);`
`save(nomdufichier,'F','X')`
sauve les variables *F* et *X* dans le fichier `nomdufichier`, la commande `string` permet de transformer un nombre en une chaîne de caractères.
- `load nomdufichier`
charge les données qui sont dans le fichier (Scilab affecte à nouveau automatiquement les variables *F* et *X*).
- La commande `savematfile` permet de sauver les variables dans un format utilisable dans le logiciel MATLAB.
- `clear`
efface toutes les variables et données existantes.

5 Graphisme dans Scilab

5.1 Graphe de points en 2D

Soient Y et Z deux vecteurs de taille n .

- Définir une courbe, première possibilité :
`plot(Z)` relie les points de coordonnées $(i, Z(i))$.
- Définir une courbe, deuxième possibilité :
`plot(Y,Z)` relie les points de coordonnées $(Y(i), Z(i))$.
- Définir deux ou plusieurs courbes, première possibilité :
`plot(Y,Z,Y,cos(Z))`, graphes des points de coordonnées $(Y(i), Z(i))$ et de coordonnées $(Y(i), \cos Z(i))$.
- Définir le style des courbes :
`plot(Y,Z,'-r',Y,cos(Z),':')` on peut pour chacune des courbes imposer un style de ligne, un style de point, une couleur.
 - Quelques styles de lignes :
 - ligne pleine, : pointillés, ...
 - Quelques couleurs :
y jaune, r rouge, b bleu, g vert, w blanc, k noir, ...
 - Quelques styles de points :
+, o, ., square, diamond, none, ...
- Contrôler les axes des courbes : la taille des axes est gérée automatiquement par Scilab. Pour imposer la taille des axes, il faut utiliser :
`a=gca(); a.data_bounds=[xmin,xmax,ymin,ymax]`.
Pour obtenir la même échelle sur les deux axes, taper `axis square`.
- Rajouter des annotations sur le graphe :
 - `title` ajoute un titre au graphe,
 - `xlabel` ajoute une légende à l'axe horizontal du graphe,
 - `ylabel` ajoute une légende à l'axe vertical du graphe,
 - `legend` ajoute des légendes aux différentes courbes.

Pour plus de précisions utiliser la commande `help plot`.

Exemple (le résultat est donné dans la figure 1) :

```
--> Y=[0:0.05:1];Z1=sin(2*%pi*Y);Z2=cos(2*%pi*Y);
--> plot(Y,Z1,'b',Y,Z2,'+k');
--> title('Exemple de courbes');
--> xlabel('Y');ylabel('Z');
--> legend('sin','cos');
```

De nombreuses autres commandes de graphisme existent (`champ`, `contour2d`, `plot2d`, ...)

5.2 Graphe d'une fonction

Le graphe d'une fonction sur un intervalle I se ramène au graphe d'un vecteur, en choisissant une discrétisation de cet intervalle I .

Exemple :

```
-->x=linspace(0,%pi,100);
-->plot(x,sin(x));
-->plot(x,f2(x,1));
```

5.3 Graphe d'une courbe dans l'espace

Soient X , Y et Z trois vecteurs de taille n . Il s'agit donc d'une courbe paramétrique.

`param3d(X,Y,Z)` donne le graphe des points $(X(i), Y(i), Z(i))$

5.4 Graphe d'une surface

Soient X et Y deux vecteurs de tailles n et m , et Z une matrice de taille $n \times m$ (resp. soit X , Y et Z trois matrices de taille $n \times m$).

La génération des surfaces se fait à l'aide des deux commandes `surf` et `mesh`.

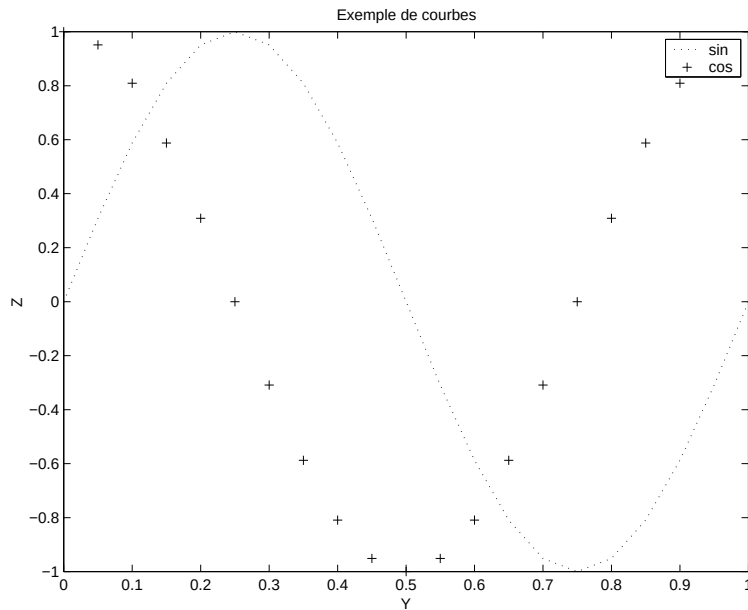


FIGURE 1 – La figure obtenue avec l'exemple précédent

- `surf(Z)` trace une surface « pleine » passant par les points de coordonnées $(i, j, Z(i, j))$.
- `surf(X,Y,Z)` trace une surface « pleine » passant, si X et Y sont des vecteurs, par les points $(X(i), Y(j), Z(i, j))$ ou si X et Y sont des matrices par les points $(X(i, j), Y(i, j), Z(i, j))$.
- `mesh(Z)` trace un maillage du style « fil de fer » passant par les points $(i, j, Z(i, j))$.
- `mesh(X,Y,Z)` trace un maillage (« fil de fer ») passant, si X et Y sont des vecteurs, par les points $(X(i), Y(j), Z(i, j))$ ou si X et Y sont des matrices par les points $(X(i, j), Y(i, j), Z(i, j))$.
- `contour(Z)` trace les courbes de niveaux de la surface passant par les points $(i, j, Z(i, j))$.
- Beaucoup d'autres possibilités existent ... voir l'aide en ligne.

Remarque 5.1 La gestion des axes et des annotations graphiques est identique à celle du graphisme 2-D.

Exemple : Voici le tracé d'une surface $z = f(x, y)$ sur $[0, 1] \times [0, 1]$

```
function [z]=ma_fonction(x,y)
    z=cos(x+y^2);
endfunction

x=linspace(0,1,100);
y=x;

surf(x,y,feval(x,y,ma_fonction));
f=gcf();
f.color_map=hotcolormap(32);
```

Notez l'utilisation de la routine `feval` qui permet d'évaluer une fonction sur toute une famille de points de façon beaucoup plus concise et efficace qu'en utilisant des boucles.

La commande `f.color_map=hotcolormap(32)` permet de changer la palette de couleurs. La variable `f` définie par la commande `gcf()` désigne un 'pointeur' vers la dernière figure ouverte.

5.5 Graphe d'une fonction de deux variables réelles ou d'une surface paramétrée

De même que pour le graphisme 2D, on se ramène au graphe d'une courbe ou de matrices :

- Courbe dans l'espace : $t \rightarrow (x(t), y(t), z(t))$ pour $t \in I$. On se donne une discrétisation de I : T vecteur de taille n et on se ramène au graphe des vecteurs $(x(T), y(T), z(T))$.
- Surface dans l'espace donnée par $(x, y) \rightarrow z(x, y)$ pour $x \in I, y \in J$. On se donne une discrétisation de I et de J : I_x vecteur de taille n , J_y vecteur de taille m et on se ramène au graphe de la matrice $z(I_x, J_y)$.

- Surface paramétrée donnée par $(u, v) \rightarrow (x(u, v), y(u, v), z(u, v))$ pour $u \in I, v \in J$. On se donne une discrétisation de I et de J : I_u vecteur de taille n , J_v vecteur de taille m et on se ramène au graphe des matrices $(x(I_u, J_v), y(I_u, J_v), z(I_u, J_v))$.

5.6 Gestion des figures

5.6.1 Ouvrir une figure

Toute commande graphique (`plot`, `surf`,...) ouvre une fenêtre graphique si aucune n'est encore ouverte. On peut également ouvrir une fenêtre graphique à l'aide de la commande `figure`.

5.6.2 Ouvrir plusieurs figures

Toute nouvelle figure doit être ouverte à l'aide de la commande `scf`. Les commandes graphiques sont effectuées dans la dernière fenêtre activée. Pour changer de fenêtre activée, on utilise la commande `gcf`.

5.6.3 La commande `clf`

Toute commande graphique (`plot`, `surf`,...) se superpose aux précédentes si on ne change pas de fenêtre avec la commande `figure`. Pour effacer le contenu d'une figure, il faut utiliser la commande `clf`.

5.6.4 Créer plusieurs graphes sur une même figure

- Deux graphes côte à côte

```
>>subplot(1,2,1)
%commande graphique 1
>>subplot(1,2,2)
%commande graphique 2
```

- Deux graphes l'un au-dessous de l'autre

```
>>subplot(2,1,1)
% commande graphique 1
>>subplot(2,1,2)
% commande graphique 2
```

- Quatre graphes

```
>>subplot(2,2,1)
% commande graphique 1
% graphe situé en haut à gauche
>>subplot(2,2,2)
% commande graphique 2
% graphe situé en haut à droite
>>subplot(2,2,3)
% commande graphique 3
% graphe situé en bas à gauche
>>subplot(2,2,4)
% commande graphique 4
% graphe situé en bas à droite
```

5.6.5 Sauvegarde et export d'une figure

On peut sauvegarder ou exporter une figure depuis les menus de la fenêtre de la figure. On peut également utiliser les commandes `xsave`, `xs2eps`, `xs2bmp`, `xs2gif`, etc ...

5.7 Animation de graphisme

Il faut activer l'option `pixmap` de la figure par les commandes suivantes

```
f=gcf();
f.pixmap='on';
```

A partir de ce moment là, les commandes graphiques (y compris `clf`) n'afficheront pas leur résultat dans la fenêtre mais dans le dessin se fera dans la mémoire de l'ordinateur. Pour afficher le résultat final de toutes les commandes il faut utiliser la commande `show_pixmap`.

- La commande `pause` permet d'effectuer un arrêt dans un programme, en particulier entre deux commandes `plot`. On peut alors faire afficher des variables, faire des calculs intermédiaires, etc ... Pour relancer le programme, il faut taper `resume` dans la fenêtre principale de Scilab.
- La commande `halt` effectue aussi une pause dans le programme. Pour relancer le programme, il suffit d'appuyer sur une touche quelconque.
- La commande `xpause(t)` permet d'effectuer une pause dans un programme d'une durée fixée par `t` et exprimée en micro-secondes.

Exemple :

```
f=gcf(); // On crée une nouvelle figure
f.pixmap='on'; // On active le mode pixmap
y=0:0.1:2;
for k=0:0.1:2
    clf // On efface
    plot(y,sin((y+k)*%pi),'-r'); // On trace
    show_pixmap // On affiche
    xpause(100000); // Pause de 0.1 seconde
end;
```

6 Quelques commandes utiles

Voici quelques noms de commandes qui peuvent être utiles. Nous renvoyons le lecteur à l'aide de Scilab pour plus de détails sur ces différentes fonctions.

6.1 Opérations sur les coefficients d'une matrice

- `max` Coefficient maximal.
- `min` Coefficient minimal.
- `mean` Moyenne des coefficients.
- `median` Valeur médiane des coefficients.
- `sort` Tri par ordre croissant des coefficients.
- `prod` Produit des coefficients.
- `sum` Somme des coefficients.

6.2 Propriétés élémentaires d'une matrice

- `det` Déterminant.
- `spec` Valeurs propres et vecteurs propres.
- `expm` Exponentielle de matrice.
- `svd` Décomposition en valeurs singulières.
- `inv` Matrice inverse.
- `kernel` Noyau.
- `range` Image.
- `rank` Rang.
- `sqrtn` Racine carrée d'une matrice.
- `trace` Trace.

6.3 Norme et conditionnement d'une matrice

- `cond`, `condest` Conditionnement d'une matrice.
- `norm` Normes de matrice.

6.4 Réduction classiques pour les matrices

- `chol` Factorisation de Choleski.

- `hess` Forme de Hessenberg.
- `lu` Factorisation LU.
- `qr` Décomposition QR.
- `rref` Forme échelonnée.
- `schur` Décomposition de Schur.

6.5 Interpolation numérique

- `interp1` Méthodes d'interpolation 1-D.
- `interp2` Méthodes d'interpolation 2-D.
- `interp3` Méthodes d'interpolation 3-D.
- `spline` Interpolation par spline cubique.

6.6 Intégration numérique

- `integrate`, `intg`, `int2d`, `int3d` Méthodes de quadratures numériques.
- `inttrap` Méthode des trapèzes.

6.7 Equations différentielles ordinaires

- `ode` Résolution d'EDO par différentes méthodes (adams, stiff, Runge-Kutta d'ordre 4, Runge-Kutta-Fehlberg d'ordres 4 et 5, ...)
- `odeoptions` pour configurer certains paramètres de `ode`

Pensez à regarder les demos disponibles dans Scilab pour avoir quelques illustrations de ce qu'il est possible de faire.

6.8 Equations aux dérivées partielles

- `diff` Calcul de différences.
- `numdiff` Approximation du gradient.

6.9 Algorithmes de minimisation et de recherche de racines

- `pcg` Gradient conjugué préconditionné.
- `qmr` Quasi-minimum residual préconditionné..
- `optim` Minimisation d'une fonction.

6.10 Zéros de fonctions

- `fsolve` Résolution de $f(x) = 0$.
- `poly` Définition d'un polynôme.
- `roots` Racines d'un polynôme.

6.11 Fourier

- `fft` Transformation de Fourier rapide en 1-D.
- `ifft` Inverse de la transformation de Fourier rapide en 1-D.
- `fft2` Transformation de Fourier rapide en 2-D.